# Setup: environment and API keys

Everything in this repository is API orchestration: no GPU, no local model, no cluster. A laptop with Python and network access is enough. Five steps.

## 1. Prerequisites

- Python 3.11 or newer.
- uv (the environment and lockfile manager). Install with `curl -LsSf https://astral.sh/uv/install.sh | sh` (macOS/Linux) or `pip install uv`.

## 2. Install

```
uv sync                      # creates .venv from uv.lock (7 packages: anthropic, openai,
```

Always run scripts from the repository root, as `uv run python scripts/<name>.py`. The drivers open `runs/`, `configs/` and `src/` by relative path.

## 3. API keys

Copy the template and fill in what you need:

```
cp .env.example .env         # .env is gitignored; never commit it
```

| variable | provider | needed for |
|---|---|---|
| `CLAUDE_API_KEY` | Anthropic | **every experiment** — all judges run on `claude-opus-4-8`; also the Claude targets (`haiku`, `sonnet-5`, `opus-5`, `fable-5`). Falls back to `ANTHROPIC_API_KEY` if unset. |
| `OPENAI_API_KEY` | OpenAI | the `gpt-5.6` and `gpt-5-mini` targets |
| `GEMINI_API_KEY` | Google | the `gemini-3.1-pro` and `gemini-3.6-flash` targets |
| `OPENROUTER_API_KEY` | OpenRouter | the `deepseek-v4`, `mistral-medium` and `kimi-k3` targets |

Only `CLAUDE_API_KEY` is mandatory. A run that names a target whose key is missing fails at that cell with a message naming the variable; nothing else is affected. Keys are read once at import from `.env` (`python-dotenv`); a variable already exported in the shell takes precedence. Nothing ever prints a key.

The alias -> exact model id table is `src/core/config.py` (`_REGISTRY`); the panel and the judge roles are `configs/panel.yaml`. Both are the single source of truth: never put a raw model id or a key anywhere else.

## 4. Check

```
uv run python scripts/selfcheck.py            # imports, the 20 scenario fingerprints, pay
uv run python scripts/selfcheck.py --ping     # plus one 1-token call per provider that ha
```

A `401` on the ping means the key is wrong; `404` means the key's account has no access to that model id; `429` means the key works and you are rate-limited.

## 5. A first run

The cheapest end-to-end exercise of the harness is one replay of one control cell (two agent sessions plus a few judge calls):

```
uv run python scripts/_suite_run.py --scn U10 --models haiku --arms neutral_chain --reps 1
uv run python scripts/summarize.py --rq controls
```

Then `README.md` lists the command that reproduces each experiment.

## Practical notes

- **Concurrency.** Keep in-flight agent sessions at or below ~12 per provider (`--parallel x --workers` on the confirm driver, `--workers` elsewhere); above that the providers return 429s and reps are lost to retries.
- **Cost and time.** One (scenario, model) cell is 8 replays of a two-session chain plus judge calls — roughly 16 agent sessions and 30–60 judge calls. An experiment over the full grid (20 scenarios x 11 models) is about 3,000 agent sessions; the intervening-sessions experiment is ~1,600 sessions per chain. Budget accordingly.
- **Resumability.** Every driver skips a cell that is already on disk and current; add `--force` to remeasure. A cell is written once, at the end of its run.
- **Where results land.** `runs/<experiment>/<scenario>/<model>/<arm>.json` — one JSON per cell holding every replay's full transcripts, tool calls, memory contents, judge verdicts and provenance stamps. Nothing is ever re-run to see what happened. `SUITE_ROOT=/other/dir` redirects the store.
- **Refusals.** A provider-side safety refusal is recorded as such and excluded from the denominator; the pairs the provider refused on every attempt are listed in `runs/_index/refusal_mask.json` and skipped.
- **Containment.** Every tool is fake and log-only: tool bodies mutate an in-memory dictionary and return strings. No real network egress, code execution or side effect is possible, whatever the agent does.